DAY 03 | POST-READ

OOP and Design: Recap and Beyond

What we covered today, plus ideas to stretch your understanding.

Prepared by **Ahmed** | linkedin.com/in/ahmed-al-imaad

Part A: Summary of today

Topic	What to remember
class and object	A class is a blueprint; objects are instances made from it.
__init__ and self	__init__ sets up a new object; self refers to the current object.
Inheritance	class Child(Parent). super() calls the parent. Override to change behaviour.
Pillars	Encapsulation, abstraction, inheritance, polymorphism.
Exceptions	try / except / else / finally. Catch specific errors. raise for your own rules.
SOLID	Single responsibility, open/closed, Liskov, interface segregation, dependency inversion.
Modules	Any .py file. import it. Packages are folders of modules.
NumPy	Fast array maths without loops: mean, max, filtering with masks.
Decorators	@decorator wraps a function to add behaviour.
Generators	yield produces values one at a time, saving memory.

Part B: Going beyond today

1. Dataclasses: classes without boilerplate

```
from dataclasses import dataclass

@dataclass
class Book:
    title: str
    author: str
    pages: int = 0

print(Book("Ikigai", "Garcia", 208))
```

@dataclass writes __init__, __repr__ and __eq__ for you. Pydantic models (Day 6) follow the same idea with validation added.

2. Abstract base classes

```
from abc import ABC, abstractmethod

class Payment(ABC):
    @abstractmethod
    def pay(self, amount): ...

class UPI(Payment):
    def pay(self, amount):
        return f"Paid {amount} via UPI"
```

You cannot create a Payment directly; every subclass must implement pay(). This is the open/closed and dependency inversion principles in code.

3. Composition over inheritance

Inheritance means "is a". Composition means "has a". A Car has an Engine; it is not an Engine. Prefer composition when the relationship is "has a". It keeps class trees shallow and flexible.

4. Context managers

You already used with open(). You can write your own with __enter__ and __exit__, or with contextlib.contextmanager, to guarantee clean up such as closing database connections.

5. Decorators with functools.wraps

```
from functools import wraps

def log_call(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        return func(*args, **kwargs)
    return wrapper
```

wraps keeps the original function's name and docstring, which matters for debugging and for frameworks like FastAPI.

6. NumPy broadcasting

```
import numpy as np
prices = np.array([[100, 200], [300, 400]])
print(prices * np.array([1.18, 1.05]))  # different tax per column
```

Part C: Check yourself

- What is the difference between a class and an object?
- Why should you avoid a bare except:?
- Explain the single responsibility principle with an example.
- Why is a generator better than a list for a 10 GB log file?
- What does a decorator return?

Part D: Practice ideas for tonight

- Model a Shape base class with Circle and Rectangle subclasses that each implement area().
- Write a @timer decorator that prints how long a function took.
- Write a generator that yields Fibonacci numbers below a limit.

Coming up on Day 4: charts with Matplotlib, how clients, servers and APIs work together, and storing data in MongoDB.

Otsukaresama deshita (お疲れ様でした): thank you for your hard work today